Analysis of Different Large Language Models Prompting Techniques' Efficacy in Predicting Students' Required Level of Assistance

Thien-Lac Quach[1,2][0009−0004−6310−5149] and Ton-Minh-Ky Tran[1,2][0009−0006−5027−2516]

[1] Faculty of Information Technology, University of Science, Ho Chi Minh City, Vietnam
[2] Vietnam National University, Ho Chi Minh City, Vietnam
{24125092,24125102}@apcs.fitus.edu.vn

Abstract. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Keywords:

1 Introduction

The prevalence of Large Language Models (LLMs) in education has opened up new avenues for enhancing student learning and academic support. LLMs have the potential to analyze vast amounts of educational data, identify patterns, and provide personalized recommendations to students and educators alike. In this study, we focus on leveraging the capabilities of LLMs to analyze students examination results and identify the level of assistance they require to succeed academically.

Using multiple prompting techniques, we analyze a given dataset of students' examination results, split into training and testing sets, to identify patterns and factors that contribute to academic success or failure in order to provide an appropriate level of academic support. We classify students' level of assistance required into different categories based on their performance, then instruct the LLMs to train themselves with the training sets and evaluate their performance on the testing sets.

By identifying at-risk students early on, Higher Education Institutions (HEIs) can implement strategies to improve retention rates and enhance the overall educational experience; students can also benefit from the predictions provided by the LLMs as an early-warning system. The results of this study can inform policy decisions and contribute to the development of more effective support systems for students in HEIs.

In order to determine the scope of our work, we begin by utilizing the systematic literature review framework devised by [5]. To this end, section 2 and 3 will elaborate on models background and choice, data collection, and evaluation metrics. Section 4 and 5 will describe present the methodology and experimental setup, including data preprocessing, feature selection, and model training. Finally, Section 6 will discuss the results and implications of our findings, as well as potential future research directions in this area.

2 Background

Prompting techniques presents a paradigm shift from traditional machine learning approaches, which typically require extensive feature engineering and model selection. Instead, prompting techniques leverage the inherent capabilities of LLMs to understand and generate human-like text based on the input provided. This allows for a more flexible and efficient way to extract insights from data without the need for complex model architectures or extensive training.. Below, we outline key prompting techiniques and terminologies relevant to our study:

2.1 In-context Learning

In-context learning (ICL) is the capability of LLMs to acquire new skills or knowledge directly from user-provided exemplars – an example of input-output pairs – or instructions, without the need for extensive training or fine-tuning. The

model recognizes pattern in the input data rather than relying on pre-trained weights. Two main techniques for in-context learning are zero-shot and few-shot prompting.

Zero-shot Prompting Zero-shot prompting involves providing the model with a task description or instruction without any exemplars. Numerous variants of zero-shot prompting exist since its inception, including those that combine other concepts, such as chain-of-thought prompting, or those that simply modify the content of the prompt, such as role prompting or style prompting.

Few-shot Prompting Few-shot prompting, on the other hand, involves providing the model with a few input-output pairs as examples to what the model should do. Few-shot prompting is generally harder to implement given design decisions regarding exemplars, such as the number or ordering of exemplars, or the choice of instructions. However, it has been shown to significantly improve the performance of LLMs on various tasks compared to zero-shot prompting.

2.2 Thought Generation

Thought generation is a prompting technique that encourages LLMs to generate intermediate reasoning steps or explanations before providing a final answer. This approach can help improve the model's performance on complex tasks by allowing it to break down the problem into smaller, more manageable parts. Chain-of-thought prompting is a specific type of thought generation that involves guiding the model through a series of logical steps to arrive at a solution. It is often used in conjuction with other prompting techniques, such as zero-shot or few-shot prompting, to further enhance the model's performance.

Chain-of-thought Prompting In chain-of-thought (CoT) prompting, the LLMs is encourage to break down a complex problem into a series of intermediate reasoning steps and express them before arriving at a final answer. This technique has been shown to significantly improve the performance of LLMs on tasks that require multi-step reasoning, such as mathematical problem-solving or logical inference. By guiding the model through a structured thought process, CoT prompting can help it arrive at more accurate and coherent answers.

3 Related Work

3.1 Data aggregation

It stands to reason that gathering high-quality and representative data is often a difficult endeavour. [1] delineated a framework where a list of hypothetically relevant features are enumerated and filtered via information gain. More specifically, the features that are taken into consideration include those relating to

students' study hours, participation in extracurricular activities, hours of sleep, and mock exams practiced [7]. Surveys for these features are deployed online as a questionaire via the platform Google Forms.

3.2 Performance metrics

Common metrics to assess multimodal classification algorithm include accuracy, precision, recall, F1-score [1, 9] and area under the receiver operating characteristic curve (AUC-ROC) [2, 8]. These metrics provide insights into the model's ability to correctly classify instances, handle imbalanced datasets, and make informed decisions based on the predictions [2]. Techniques for handling imbalanced datasets are crucial when working with DT-based models to avoid bias towards the majority class in the models. Here, we focus on Synthetic Minority Over-sampling Technique (SMOTE), specifically its variant ADASYN, which utilies a weighted distribution to determine the learning difficulty of minority class samples and generates synthetic data accordingly [3, 4]. This approach helps to balance the dataset and improve the performance of the decision tree models in predicting the minority class.

4 Methodology

In this section, we explain the methodology used to conduct our research. We first describe the dataset and preprocessing pipeline. Next, we outline the traditional machine learning models and LLMs we employed for our experiments. Finally, we discuss the evaluation metrics and experimental setup.

4.1 Dataset and Preprocessing

Dataset We utilized a dataset named Open University Learning Analytics dataset (hereby referred to as OULAD), described in [6]. This dataset reports the background, behavior, and performance of students from England in a given Virtual Learning Environment (VLE). The dataset contains 32,593 records of 20 features, including demographic information, course interactions, and final results. The features include both categorical and numerical data, which required preprocessing before model training. The dataset, as outlined by its authors, consists of four modules:

- **Student Demographics:** This module includes background information about the students, such as age, gender, and educational as well as socio-economic background. One notable feature is the Index of multiple deprivation (IMD), which is a measure of socio-economic status based on the students' postal codes.
- **Course Registrations:** This module contains information about the courses students are registered for, including course/presentation codes and their lengths. It also includes data students' registration and unregistration dates.

- **Assessment Results:** This module captures the students' performance in three types of assessments, namely Tutor Marked Assessments (TMA), Computer Marked Assessments (CMA), and Final Exams (Exam), adjusted by weights. It provides insights into their academic progress and outcomes.

- **VLE interactions:** This module records the students' interactions with the Virtual Learning Environment (VLE), including the number of clicks on various pages and resources. It also includes the dates of these interactions, which can be used to analyze engagement patterns over time. This module also provides information on the resources in the VLE.

The following table summarizes the tables belonging to each module and the number of records in each table:

Module	Table Name	Number of Records	Description
Student Demographics	studentInfo	32,593	Contains demographic information about students. Contains the following columns: `code_module`, `code_presentation`, `id_student`, `gender`, `region`, `highest_education`, `imd_band`, `age_band`, `num_of_prev_attempts`, `studied_credits`, `disability`, `final_result`
Course Registrations	courses	22	Contains information about courses. Contains the following columns: `code_module`, `code_presentation`, `length`
Registrations	studentRegistration	32,593	Contains information about students' registrations. Contains the following columns: `code_module`, `code_presentation`, `id_student`, `date_registration`, `date_unregistration`
Assessment Results	assessments	206	Contains information about assessments available in module-presentations. Contains the following columns: `code_module`, `code_presentation`, `id_student`, `id_assessment`,`assessment_type`, `date`, `weight`
Assessment Results	studentAssessment	173,912	Contains information about students' results in assessments. Contains the following columns: `id_student`, `id_assessment`, `is_banked`, `date_submitted`, `score`
VLE interactions	interactions	10,655,280	Contains information about students' interactions with the VLE. Contains the following columns: `code_module`, `code_presentation`, `id_student`, `id_site`, `date`, `sum_click`
VLE interactions	vle	6,364	Contains information about the resources in the VLE. Contains the following columns: `id_site`, `code_module`, `code_presentation`, `activity_type`, `week_from`, `week_to`

Table 1: Summary of the OULAD dataset tables.

To correctly and effectively classify the students' final results, preprocessing must be done to merge the tables and create a single table containing all the features, keyed according to the `id_student` column. In the next subsection, we describe the preprocessing steps taken to achieve this.

Preprocessing In the figure below, we outline the steps taken to preprocess the OULAD dataset.

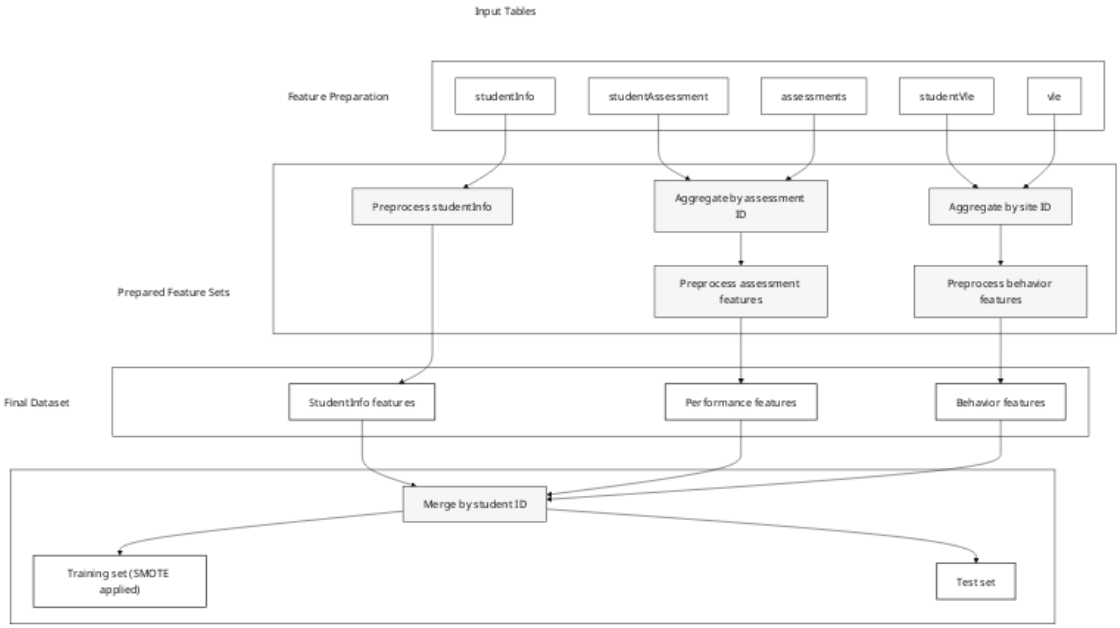

Fig. 1: Preprocessing pipeline for the OULAD dataset

We divide the tables into three groups: background (studentInfo), performance (studentAssessment and assessments), and behavior (studentVle, vle, and assessments for cutoff construction). We thus perform the following preprocessing tasks on each group as follows:

- **Background:** We first perform one-hot encoding on the categorical features in the `studentInfo` table, which include students' gender, region, highest education and disability status. For age and IMD bands, we perform ordinal encoding, as these features have an inherent order.

- **Performance:** We first merge the `studentAssessment` and `assessments` tables on `id_assessment`. Missing due dates and scores are explicitly tracked using binary indicators (`has_due_date` and `score_missing`), and missing values are imputed using the median. We then engineer timing features from submission behavior, including `submission_delay`, late/early flags, and a capped delay variable based on the 1st and 99th percentiles to reduce outlier effects. In addition, we compute weighted performance (`weighted_score`) using assessment weights and its min-max normalized version. Finally, we aggregate all assessment records at the student-module-presentation level (`id_student`, `code_module`, `code_presentation`) to produce summary statistics (counts, central tendency, variability, weighted-score totals, lateness ratios, delay average, and missingness ratios), and append assessment-type frequency features via a pivot table.
- **Behavior:** To construct behavior features from VLE interactions, we first derive a module-presentation cutoff date as the latest assessment date and keep only clicks that occur on or before this cutoff, preventing post-assessment leakage. We then merge `studentVle` with `vle` to attach `activity_type` for each `id_site` (with missing types labeled as unknown). Next, we aggregate `sum_click` values per student-module-presentation and activity type, and pivot them into wide features (e.g., `clicks_forum`, `clicks_resource`). Because interactions are processed in chunks for scalability, we finally sum partial chunk-level results to obtain one behavior-feature row per (`id_student`, `code_module`, `code_presentation`).

After preprocessing, we aggregate three groups of features, using `student_id` as the key, to create the final dataset for modeling. This dataset contains 32,593 records and 72 features, including the students' ID and their final results. The final results are categorized into four classes: `Distinction`, `Pass`, `Fail`, and `Withdrawn`.

Next, we split the dataset into training and testing sets, with an 80-20 split. We use `sk_learn`'s `SMOTE` to address class imbalance in the training set, which is a common issue in educational datasets where certain outcomes (e.g., withdrawal) may be more prevalent than others. This increases the size of the train set to 39,556 records, which raises the total dataset size to 46,076 records.

4.2 Modeling

For this research, we utitlize `sk_learn`'s implementations of the following traditional machine learning models: Random Forest, Decision Tree, Support Vector Machine, Gradient Boosting, Gaussian Naive Bayes, and K-Nearest Neighbors. The parameters for each models are obtained by running `sk_learn`'s `RandomizedSearchCV` on the models and the dataset and presented in the following table:

Model	**Parameters**
Random Forest	`n_estimators=300`, `max_depth=50`, `random_state=0`, `min_samples_split=5`, `min_samples_leaf=2`, `max_features='sqrt'`, `bootstrap=False`
Decision Tree	`random_state=0`, `min_samples_split=20`, `min_samples_leaf=4`, `max_depth=10`, `criterion='gini'`, `splitter='best'`
Gradient Boosting	`n_estimators=100`, `subsample=1`, `learning_rate=0.2`, `min_samples_split=2`, `min_samples_leaf=4`, `max_depth=5`, `max_features='auto'`
Support Vector Machine	`StandardScaler()` + `LinearSVC(C=0.785, class_weight='balanced', random_state=0, max_iter=5000, dual='auto')`
Gaussian Naive Bayes	`var_smoothing=9.22e-12`
K-Nearest Neighbors	`StandardScaler()` + `KNeighborsClassifier(n_neighbors=7, weights='uniform', p=1, metric='minkowski', leaf_size=50)`

Table 2: Hyperparameters used for each traditional machine learning model.

For baseline LLMs, we utilize two models available on Huggin Face: `qwen-2.5-3b` and `llama-2-8b`. The models are not fine-tuned and was prompted to perform the classification task with zero-shot prompting. The prompt used is as follows:

Put the prompt here.

5 Results

5.1 Traditional Predictive Models

Various traditional model architectures are implemented. Of particular are Random Forest, Decision Tree, Gradient Boosting, Support Vector Machine, Gaussian Naive Bayes and K-Nearest Neighbors. The performances of each model is measured using plain accuracy, balanced accuracy, and F1 score. The accuracy of a model can be calculated as follows:

$$\text{Accuracy} = \frac{\text{True Positive} + \text{True Negative}}{\text{Total Prediction}}.$$

It is an intuitive measure of how correct the model predictions are, but falls short when the given dataset is imbalanced. To mitigate this issue, the balanced accuracy is preferred:

$$\text{Balanced Accuracy} = \frac{\text{Sensitivity} + \text{Specificity}}{2}.$$

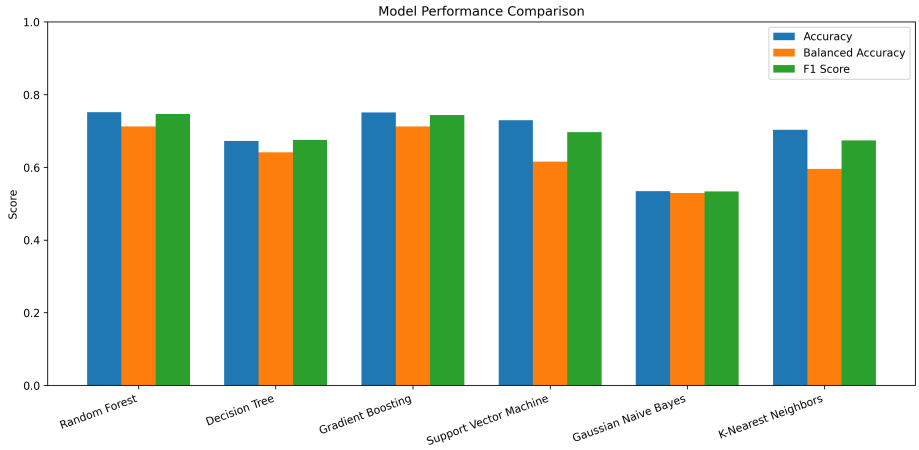

Fig. 2: Performance comparison of traditional predictive models

In addition, the F1 score is also utilized, calculated as the harmonic mean of precision and recall:

$$\text{F1 Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

As shown in Fig. 1, Gradient Boosting consistently outperforms other predictive models.

5.2 Large Language Models

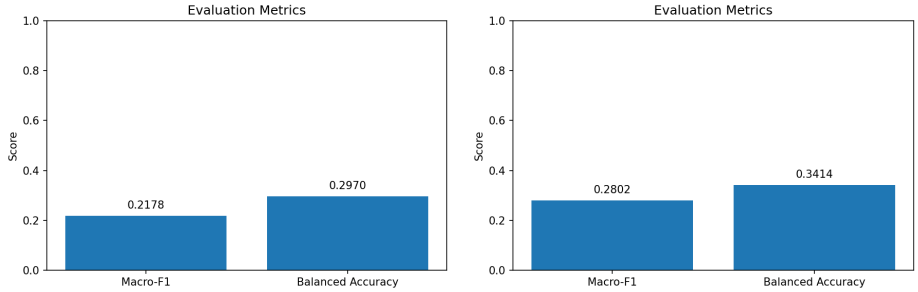

Fig. 3: Performance comparison of Qwen2.5-3B and Meta-Llama3-9B

We use a modified metric for LLMs—Macro F1. This metric effectively treats each class in a model equally important by taking the mean of the F1 scores of

each class. Observe that it performs noticeably worse than the lowest scoring traditional predictive model.

References

1. Alias, M.A.H., Abdul Aziz, M.A., Hambali, N., Taib, M.N.: Student performance classification: a comparison of feature selection methods based on online learning activities. International Journal of Electrical and Computer Engineering (IJECE) **14**(4), 4675 (2024). https://doi.org/10.11591/ijece.v14i4.pp4675-4685
2. Brzezinski, D., Stefanowski, J.: Prequential AUC: properties of the area under the ROC curve for data streams with concept drift. Knowledge and Information Systems **52**(2), 531–562 (2017). https://doi.org/10.1007/s10115-017-1022-8
3. Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P.: SMOTE: Synthetic Minority Over-sampling Technique. (2011). https://doi.org/10.48550/ARXIV.1106.1813
4. Elreedy, D., Atiya, A.F.: A Comprehensive Analysis of Synthetic Minority Over-sampling Technique (SMOTE) for handling class imbalance. Information Sciences **505**, 32–64 (2019). https://doi.org/10.1016/j.ins.2019.07.070
5. Keathley-Herring, H., Van Aken, E., Gonzalez-Aleu, F., Deschamps, F., Letens, G., Orlandini, P.C.: Assessing the maturity of a research area: bibliometric review and proposed framework. Scientometrics **109**(2), 927–951 (2016). https://doi.org/10.1007/s11192-016-2096-x
6. Kuzilek, J., Hlosta, M., Zdrahal, Z.: Open University Learning Analytics dataset. Scientific Data **4**(1) (2017). https://doi.org/10.1038/sdata.2017.171
7. Suleiman, I.B., Okunade, O.A., Dada, E.G., Ezeanya, U.C.: Key factors influencing students' academic performance. Journal of Electrical Systems and Information Technology **11**(1) (2024). https://doi.org/10.1186/s43067-024-00166-w
8. Yang, Z., Zhang, T., Lu, J., Zhang, D., Kalui, D.: Optimizing area under the ROC curve via extreme learning machines. Knowledge-Based Systems **130**, 74–89 (2017). https://doi.org/10.1016/j.knosys.2017.05.013
9. Zaky, U., Naswin, A., Sumiyatun, S., Murdiyanto, A.W.: Performance Analysis of the Decision Tree Classification Algorithm on the Water Quality and Potability Dataset. Indonesian Journal of Data and Science **4**(3), 145–150 (2023). https://doi.org/10.56705/ijodas.v4i3.113